# String Matching Algorithms

# String Matching Introduction

- String Matching Algorithm, also known as String Searching Algorithm, is a vital class of string algorithms. It refers to methods used to find the location(s) where one or more substrings occur within a larger string.

- Given a text array, T [1.....n], of n character and a pattern array, P [1.....m], of m characters. The problems are to find an integer s, called **valid shift** where $0 \leq s < n-m$ and T [s+1.....s+m] = P [1.....m].

- In other words, to find even if P in T, i.e., where P is a substring of T. The item of P and T are character drawn from some finite alphabet such as {0, 1} or {A, B .....Z, a, b..... z}.

# Algorithms used for String Matching:

- There are different types of method is used to finding the string
  - The Naive String Matching Algorithm
  - The Rabin-Karp-Algorithm
  - The Knuth-Morris-Pratt Algorithm

# The Naive String Matching Algorithm

- The naive approach tests all the possible placement of Pattern P [1.....m] relative to text T [1.....n].
- We try shift s = 0, 1.....n-m, successively and for each shift s.
- Compare T [s+1.....s+m] to P [1.....m].
- The naive algorithm finds all valid shifts using a loop that checks the condition P [1.....m] = T [s+1.....s+m] for each of the n - m +1 possible value of s.

```
NAIVE-STRING-MATCHER (T, P)
1. n ← length [T]
2. m ← length [P]
3. for s ← 0 to n -m
4. do if P [1.....m] = T [s + 1....s + m]
5. then print "Pattern occurs with shift" s
```

- **Analysis:** This for loop from 3 to 5 executes for n-m + 1(we need at least m characters at the end) times and in iteration we are doing m comparisons.
- So the total complexity is O (n-m+1).

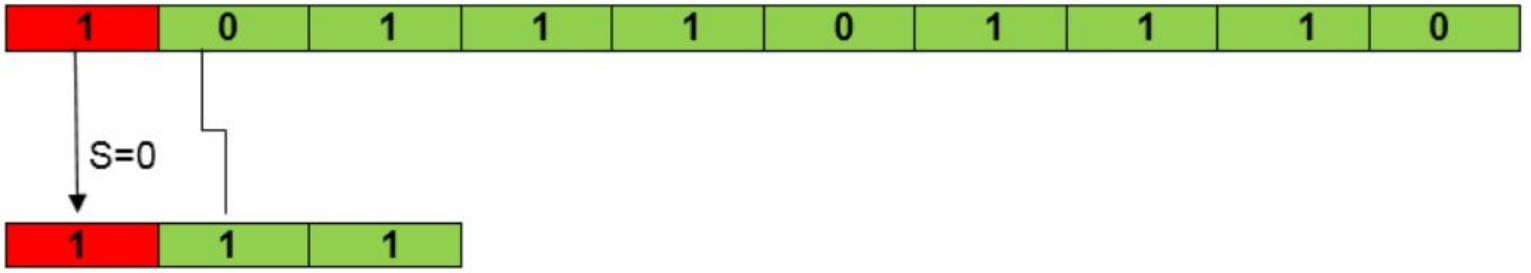
# Example:

Suppose T = 1011101110

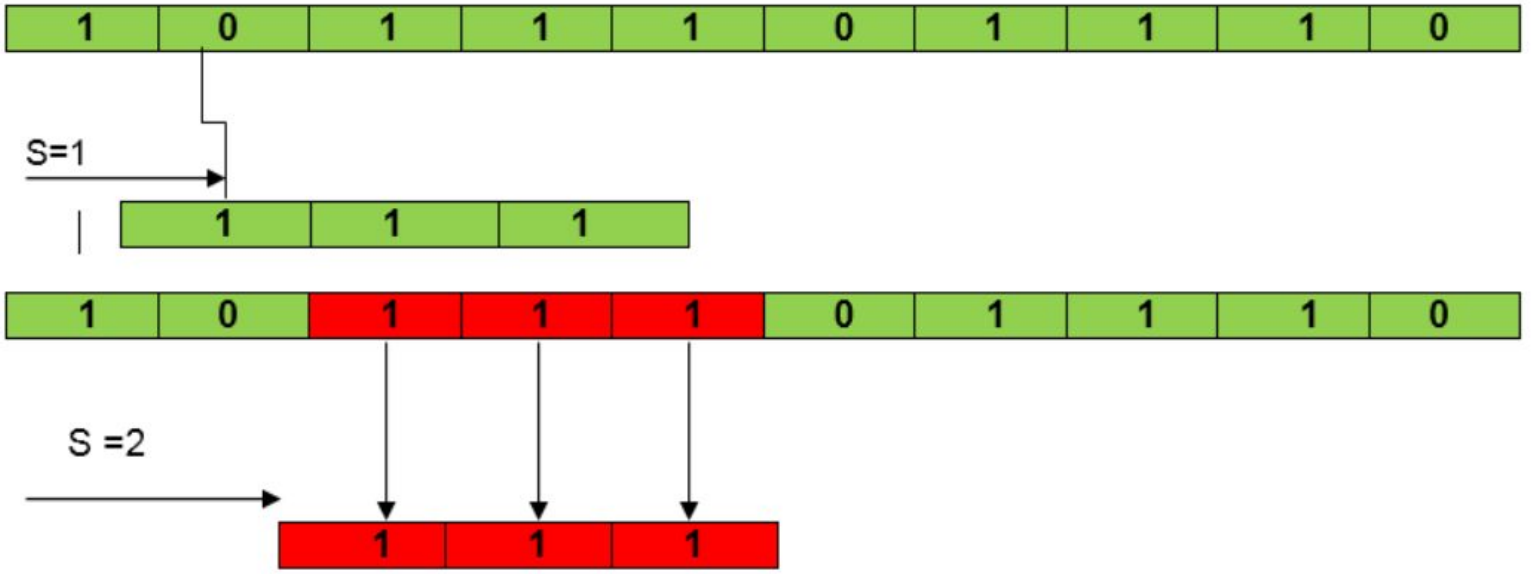
P = 111

Find all the Valid Shift

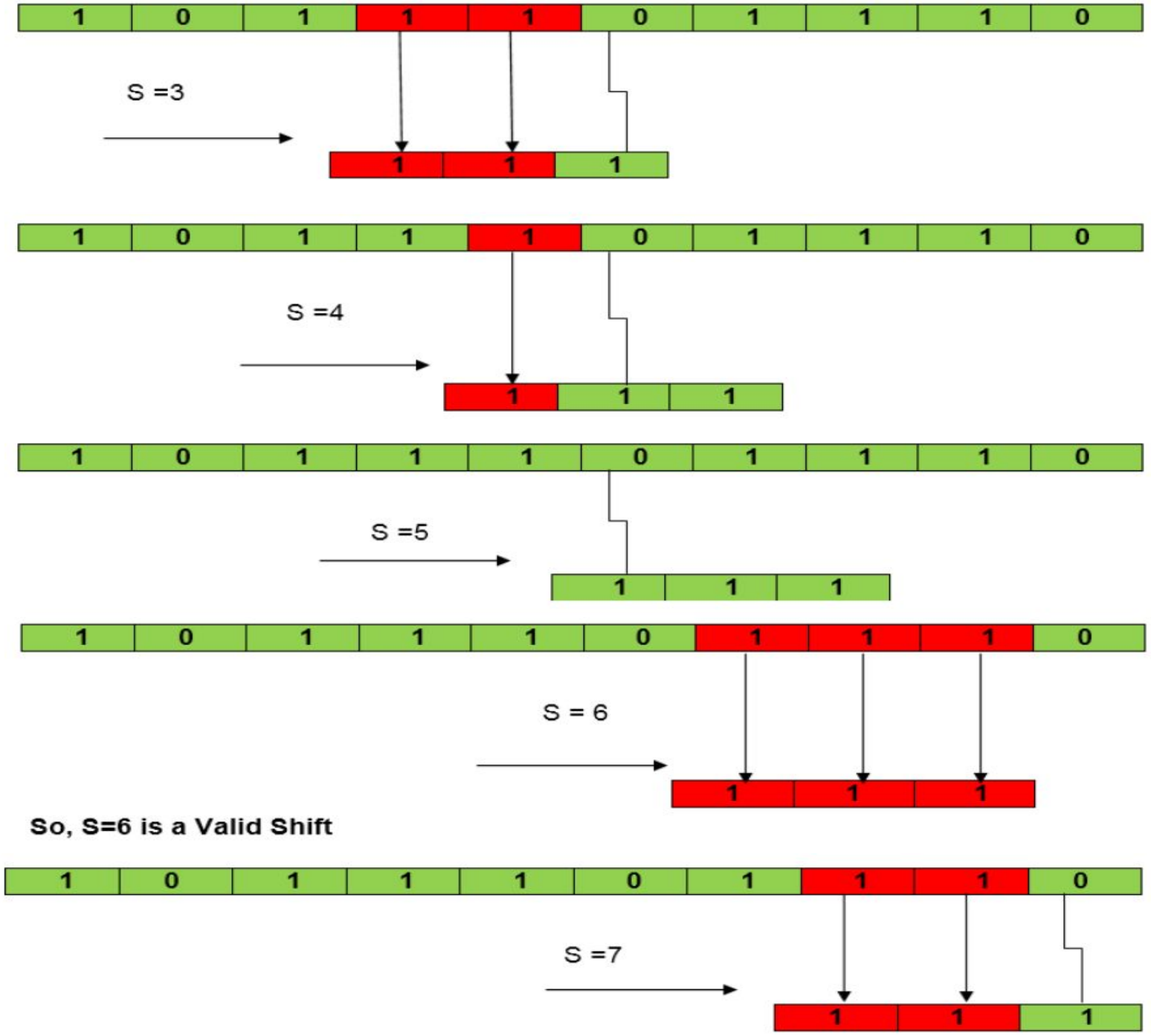
T = Text




P = Pattern




So, S=2 is a Valid Shift


1 0 1 1 1 0 1 1 1 0
S =3
1 1 1
1 0 1 1 1 0 1 1 1 0
S =4
1 1 1
1 0 1 1 1 0 1 1 1 0
S =5
1 1 1
1 0 1 1 1 0 1 1 1 0
S = 6
1 1 1
So, S=6 is a Valid Shift
1 0 1 1 1 0 1 1 1 0
S =7
1 1 1

# The Rabin-Karp Algorithm

- The Rabin-Karp Algorithm is an algorithm utilized for searching/matching patterns in the text with the help of a hash function.
- Unlike the Naïve String-Matching algorithm, it doesn't traverse through every character in the initial phase.
- It filters the characters that do not match and then performs the comparison.
- A hash function is a utility to map a larger input value to a smaller output value. This output value is known as the Hash value.
- The Rabin-Karp Algorithm is utilized to find all the occurrences of a provided pattern 'P' in a given string 'S' in O(Ns + Np) time, where 'Ns' and 'Np' are the lengths of 'S' and 'P', respectively.

# The Rabin-Karp Algorithm

- Rabin-Karp is a **string matching algorithm** that uses **hashing** to find a pattern P of length m in a text T of length n.
- Instead of comparing characters one by one at each position, it:

1. Computes a **hash** of the pattern.
2. Slides a window of size m over the text.
3. Computes the **hash** of each substring.
4. If the hashes match, it does a **character-by-character check** to confirm (to avoid spurious hits).

# The Rabin-Karp Algorithm

- Let us consider the following example to understand it in a better way.
- Suppose a string **S = "pabctxjabcjfsabcd"** and pattern **P = "abc"**. We are required to find all the occurrences of 'P' in 'S'.
- We can see that "abc" is occurring in "pabctxjabcjfsabcd" at three positions. Thus, we will return that the pattern 'P' is occurring in string 'S' at indices 1, 7, and 13.

# Hash

*A string is converted into a numeric hash using a polynomial rolling hash. For a string s of length n, the hash is computed as:*

*=> hash(s) = (s[0] * $p^{(n-1)}$ + s[1] * $p^{(n-2)}$ + ... + s[n-1] * $p^{0}$ ) %mod*

*Where:*

- *s[i] is the numeric value of the i-th character ('a' = 1, 'b' = 2, ..., 'z' = 26)*
- *p is a small prime number (commonly 31 or 37)*
- *mod is a large prime number (like 1e9 + 7) to avoid overflow and reduce hash collisions*

# Understanding the Working of the Rabin-Karp Algorithm

- A sequence of the characters is taken and checked for the possibility of the presence of the given string. If the possibility is found, then character matching is performed.
- Let us consider the following steps to understand the algorithm:

**Step 1:** Suppose that the given text string is:


**Figure 1:** The Given Text String

and the pattern to be searched in the above text string is:


**Figure 2:** The Pattern

**Step 2:** Let us start by assigning a numerical value (v)/weight for the characters we will use in the problem. Here, we have selected the first ten alphabets only (i.e., A to J).


**Figure 3:** Text Weights

**Step 3:** Let x be the length of the pattern, and y be the length of the text string. Here, we have y = 10 and x = 3. Moreover, let n be the number of characters present in the input set. Since we have taken input set as {A, B, C, D, E, F, G, H, I, J}. Therefore, n will be equal to 10. We can assume any preferable value for n.

**Step 4:** Let us now start calculating the hash value of the pattern:

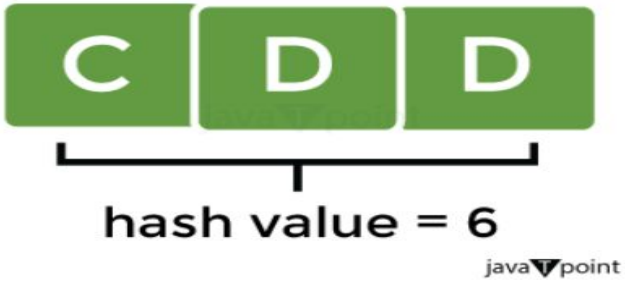



**Figure 4:** Hash Value of the Pattern

$$\text{Hash Value of Pattern (P)} = \Sigma(v * ny - 1) \bmod 13$$
$$= ((3 * 10^2) + (4 * 101) + (4 * 10^0)) \bmod 13$$
$$= 344 \bmod 13$$
$$= 6$$

**Step 5:** We will now calculate the hash value for the text window of size y.

*For the first window ABC,*

*Hash Value of text string (s) = Σ(v * nx - 1) mod 13*

*= ((1 * $10^2$) + (2 * 101) + (3 * $10^0$)) mod 13*

*= 123 mod 13*

*= 6*

**Step 6:** We will now compare the hash value of the pattern with the hash value of the text string. If they match, we will perform the character-matching operation.

In the above examples, the hash value of the first window (i.e., s) matches with the hash value of the pattern (i.e., P). Therefore, we will start matching the characters between ABC and CDD. Since the pattern does not match the first window, we will move to the next window.

**Step 7:** We will calculate the hash value of the next window by subtracting the first term and adding the next term, as shown below:

*s = ((1 * $10^2$) + ((2 * $10^1$) + (3 * $10^0$)) * 10 + (3 * $10^0$)) mod 13*

*= 233 mod 13*

*= 12*

We will use the former hash value in the following way in order to optimize this process:

$$s = ((n * (t - v[\text{character to be eliminated}] * h) + v[\text{character to be included}]) \bmod 13$$

$$= ((10 * (6 - 1 * 9) + 3) \bmod 13$$

$$= 12$$

$$\text{Where, } h = n^{y-1} = 10^{3-1} = 100$$

**Step 8:** For BCC, the hash value, s, will become 12, which is not equal to the hash value of the pattern, P, i.e., 6. Therefore, we will move to the next window. After performing some searches, we will manage to find the match for the window CDD in the text string.

**Figure 5:** Hash Value of Different Windows

# Time complexity

- The Rabin-Karp algorithm has an average-case time complexity of O(n + m) and a worst-case time complexity of O(nm), where *n* is the length of the text and *m* is the length of the pattern.
- Best/Average Case: O(n + m)

  - This occurs when hash collisions are rare.

  - The algorithm computes the hash of the pattern and compares it with rolling hashes of substrings in the text.

  - If hashes differ, it skips character-by-character comparison, making it fast.
- Worst Case: O(nm)

  - Happens when hash collisions are frequent.

  - Every hash match triggers a full character comparison, reverting to brute-force behavior.

  - This is common with poor hash functions or adversarial inputs.

# Application

- Efficient for multiple pattern matching: You can hash all patterns and compare against rolling hashes in the text.

- Great for applications like plagiarism detection, DNA sequence analysis, and document scanning.